

YOLOv8 Real-Time Object Detection Using Webcam

1. Introduction

This document explains a Python program that performs **real-time object detection** using a webcam. The application uses **Ultralytics YOLOv8**, **OpenCV**, and **CVZone** to detect objects, draw bounding boxes, display class names with confidence scores, and calculate FPS (Frames Per Second).

This project is suitable for: - Computer Vision learning - AI / Deep Learning demonstrations - Real-time surveillance or monitoring systems

2. Technologies & Libraries Used

Library	Purpose
ultralytics	Loads and runs YOLOv8 model
opencv-python (cv2)	Camera access & image processing
cvzone	Drawing stylish bounding boxes & text
math	Confidence score rounding
time	FPS calculation

3. Webcam Initialization

```
cap = cv2.VideoCapture(0)
cap.set(3, 1280) # Width
cap.set(4, 720)  # Height
```

- Opens the default webcam (index `0`)
 - Sets camera resolution to **1280×720**
-

4. YOLOv8 Model Loading

```
model = YOLO(r"D:\AI Training\Assignment\Deep-Learning\object-detection-yolo8\yolo-basics\yolov8l.pt")
```

- Loads the **YOLOv8 Large model (yolov8l)**
 - Pre-trained on the **COCO dataset**
 - Provides high accuracy for object detection
-

5. Class Names (COCO Dataset)

```
classNames = ["person", "bicycle", "car", ... "toothbrush"]
```

- Contains **80 object classes** supported by COCO
- Used to map detected class IDs to readable names

Example: - 0 → person - 2 → car - 67 → cell phone

6. Frame Processing Loop

```
while True:  
    success, img = cap.read()
```

- Continuously reads frames from the webcam
 - `img` contains the current video frame
-

7. Object Detection

```
results = model(img, stream=True)
```

- Runs YOLOv8 on each frame
 - `stream=True` improves performance for video input
-

8. Bounding Box Extraction

```
for r in results:  
    boxes = r.bboxes
```

Each detected object contains: - Bounding box coordinates - Confidence score - Class ID

9. Bounding Box Drawing

```
x1, y1, x2, y2 = box.xyxy[0]  
w, h = x2 - x1, y2 - y1  
cvzone.cornerRect(img, (x1, y1, w, h))
```

- Converts bounding box coordinates to integers
 - Calculates width & height
 - Draws **corner-style rectangles** using CVZone
-

10. Confidence & Label Display

```
conf = math.ceil((box.conf[0] * 100)) / 100  
cls = int(box.cls[0])  
cvzone.putTextRect(img, f'{classNames[cls]} {conf}', (x1, y1))
```

- Confidence score rounded to **2 decimal places**
- Displays:
 - Object name
 - Detection confidence

Example Output:

```
person 0.87  
car 0.92
```

11. FPS (Frames Per Second) Calculation

```
new_frame_time = time.time()
```

```
if prev_frame_time > 0:
    fps = 1 / (new_frame_time - prev_frame_time)
else:
    fps = 0

prev_frame_time = new_frame_time
```

- Measures real-time performance
- FPS indicates how fast frames are processed
- Printed in the console

12. Display Output Window

```
cv2.imshow('image', img)
cv2.waitKey(1)
```

- Displays real-time detection results
- `waitKey(1)` ensures smooth video playback

13. Program Output

✓ Live webcam feed ✓ Object detection with bounding boxes ✓ Class names & confidence scores ✓
Real-time FPS calculation

14. Applications

- Smart surveillance systems
- Traffic monitoring
- AI learning & demonstrations
- Industrial safety monitoring

15. Conclusion

This YOLOv8-based application demonstrates efficient and accurate real-time object detection using Python. By combining YOLOv8 with OpenCV and CVZone, the system provides visually appealing results with strong performance.

Author: Manikandan P

Technology: Python | YOLOv8 | OpenCV | AI / Deep Learning